

Hybrid LSTM-Transformer for RONIN Heading Estimation

Model

Model Name	Hybrid LSTM-Transformer
Name	Gopal Datt Bhatt

4 Section 1 — Architecture

4.1 1.1 Model Description

This model takes a window of inertial and magnetic sensor data with shape (200, 9). A stacked LSTM first extracts temporal motion features from the sequence, then a Transformer encoder processes the LSTM feature sequence to capture longer-range dependencies across the full one-second window. The final fused representation is passed through fully connected layers to predict $\sin(\text{heading})$ and $\cos(\text{heading})$.

4.2 1.2 Layer Structure

Component	Details
Input	(200 x 9)
LSTM Section	2-layer LSTM, hidden size 128, dropout 0.3
Transition Point	Full LSTM output sequence (batch, 200, 128) is linearly projected to transformer dimension 128
Transformer Section	Positional encoding used; 2 encoder layers; 4 attention heads; model dimension 128; feedforward dimension 256; dropout 0.2
How outputs combined	Mean pooling is applied to the transformer output and concatenated with the final LSTM hidden state before the fully connected block
Fully Connected Output	256 -> 64 -> 2 output neurons [sensor window → LSTM/Transformer features → fused 256 features → dense layer → 64 features → final 2 heading values]

4.3 1.3 Trainable Parameters

Total trainable parameters: 501,314.

4.4 1.4 Architectural Choices and Justification

A hybrid design was chosen to combine two useful properties. The LSTM front-end is well suited to extracting local temporal motion patterns from IMU and magnetometer sequences, while the Transformer encoder can model broader context across the full window using self-attention and positional encoding. This follows the common sequence-modelling rationale that recurrent layers capture short-range sequential structure and Transformer layers add longer-range contextual reasoning.

The LSTM hidden size was set to 128 and the LSTM depth was kept to two layers so that the recurrent encoder could learn richer motion dynamics. The Transformer dimension was kept at 128 and four

attention heads were used so that self-attention could examine the sequence from multiple subspaces while still dividing the model dimension cleanly. Two encoder layers and a feedforward dimension of 256 were used to provide more representational capacity, while the 256 -> 64 -> 2 output head remained compact enough to fuse the recurrent and attention-based summaries before predicting the circular heading target.

5 Section 2 — Training Setup

5.1 2.1 Loss Function

The loss function used was mean squared error (MSE) on the two output values [sin(heading), cos(heading)]. This is appropriate because heading is a circular quantity, and predicting sine and cosine avoids the discontinuity between 0 degrees and 360 degrees. Two angles that are close to each other remain close in the sine-cosine space even when they cross the wraparound boundary.

5.2 2.2 Training Hyperparameters

Hyperparameter	Value
Optimizer	Adam
Learning rate	0.001
Batch size	128
Number of epochs	20

The model was trained using the Adam optimizer with a learning rate of 0.001. A batch size of 128 was used to balance model throughput and memory demands for CPU execution. The maximum number of training epochs was set to 20, with early stopping applied based on validation loss to avoid unnecessary training and reduce overfitting.

5.3 2.3 Learning Rate Scheduling

Learning rate scheduling was used through ReduceLROnPlateau based on validation loss. The scheduler used a factor of 0.5, patience of 3 epochs, and a minimum learning rate of 1e-6. The learning rate remained at 0.001 during the first seven epochs and was reduced to 0.0005 from epoch 8 onward after validation improvement slowed. This helped the model continue training with smaller and more stable parameter updates.

5.4 2.4 Early Stopping

Yes. Early stopping was used with patience 7 and the monitored metric was validation loss. In the executed run, training printed “Early stopping triggered” after epoch 11, before reaching the maximum epoch limit of 20.

5.5 2.5 Training Difficulties

The main difficulty was training time. The notebook was run on CPU, and each epoch was slow because the model combines a deeper recurrent encoder with a Transformer encoder on a large number of overlapping windows. Overfitting was also a concern: the training loss kept decreasing, but the

validation loss improved only in the early epochs and then flattened or worsened. Dropout, learning-rate reduction on plateau, and early stopping were used to control this.

6 Section 3 — Results

6.1 3.1 Quantitative Results

Metric	Value
MAE on test set (degrees)	66.1114
RMSE on test set (degrees)	82.8781
Approximate training time	20,365.00 seconds (~5.66 hours)
Average validation time	168.10 seconds per epoch
Test time	134.65 seconds

6.2 3.2 Predicted vs Ground Truth Trajectory

The figure below shows model predictions against ground truth for one full unseen test trajectory, sequence a006_2. The plot covers the full sequence and contains 3,475 window-level predictions across the trajectory.

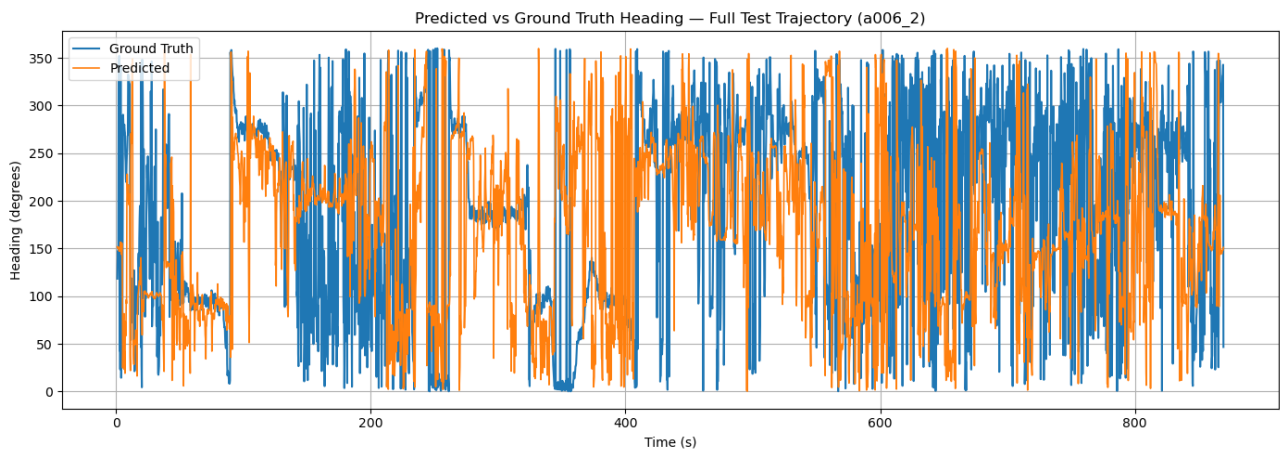


Figure 1. Predicted vs ground truth heading over one full test trajectory (a006_2).

6.3 3.3 Loss Curve

The training and validation loss curves from the executed notebook are shown below. The run completed 11 epochs before early stopping.

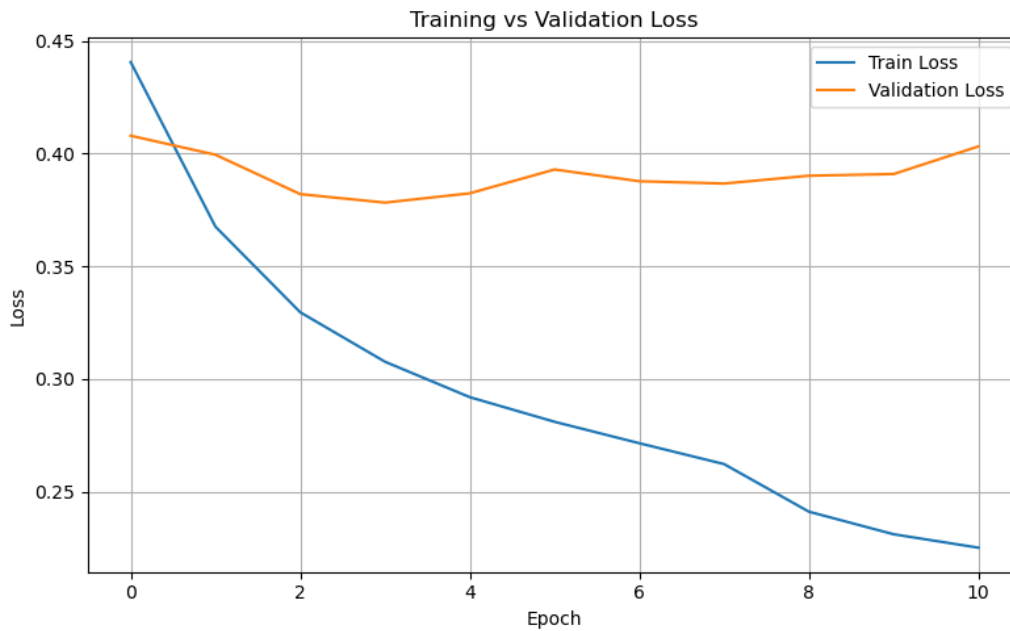


Figure 2. Training and validation MSE loss over 11 epochs.

6.4 3.4 MAE Breakdown by Scenario

Scenario	MAE (degrees)
Straight walking	58.7827
Sharp turns	89.8205
Short trajectories	64.8144
Long trajectories	66.9763

Scenario-wise MAE was computed using circular heading error on the unseen test set. Straight walking was defined as windows with small ground-truth heading change, while sharp turns were defined as windows with large ground-truth heading change. Short and long trajectories were separated using a median split based on trajectory length in windows. The median trajectory length was 2551.5 windows, so trajectories with 2551 windows or fewer were treated as short and trajectories with 2552 windows or more were treated as long.

7 Section 4 — Observations and Technical Reflection

7.1 4.1 Where the Model Performs Well

The model performs relatively better in portions of the trajectory where heading changes are smoother and the target stays within a narrower angular range for a short period. This is consistent with the scenario-wise results, where straight walking produced the lowest MAE at 58.7827 degrees. In these segments, the recurrent encoder can exploit local temporal consistency and the Transformer can reinforce stable contextual patterns across nearby windows.

7.2 4.2 Where the Model Struggles

The model struggles on difficult parts of the full unseen test trajectory where heading changes rapidly, wraps around the circular boundary, or appears noisy and unstable. This is consistent with the sharp-

turn MAE of 89.8205 degrees, which is much higher than the straight-walking MAE. In the full-trajectory plot, there are many sections where the predicted curve does not align closely with the ground truth, especially during abrupt changes and highly variable regions.

7.3 4.3 Loss Curve Interpretation

The loss curve shows clear learning at the start of training. Training loss decreases steadily from about 0.441 to 0.225. Validation loss also improves early, falling from about 0.408 to its best region around 0.378 by roughly epoch 4. After that, training loss continues to fall while validation loss stops improving and drifts upward. This suggests moderate overfitting rather than underfitting. The model has enough capacity to fit the training windows, but its generalisation to unseen subjects remains limited.

7.4 4.4 Architecture vs Training — Root Cause of Limitations

The observed limitations likely come from both the architecture and the training conditions. The heavier architecture is expressive enough to learn the training set, but the remaining test errors show that it is still not fully robust on unseen trajectories. Part of the problem may be architectural: mean-pooled Transformer summaries and a compact fusion head may still miss some heading dynamics. Part of the problem is also training-related: the model was trained on CPU, which restricted experimentation and made it harder to explore stronger regularisation or multiple tuned runs. The evidence from the loss curve suggests that training fit improved faster than validation performance, so generalisation remains the key limitation.

7.5 4.5 One Change I Would Make

If more time were available, the single most useful change would be to keep the heavier architecture but train and tune it on GPU. Faster training would make it practical to test multiple random seeds, adjust regularisation, and compare better fusion strategies while preserving the added representational capacity of the larger LSTM-Transformer design.